React useContext Hook - Detailed Guide

The **useContext** hook in React allows components to access global values without the need for prop drilling. It works with the React Context API to **create context**, **provide context**, and **consume context**.

Steps of using useContext:

1. Create Context: Use createContext() to define a new context.
2. Provide Context: Wrap components inside Context.Provider and pass values.
3. Consume Context: Use useContext(ContextName) to access values inside child components.

■ Simple Example (Theme + Counter):

```
// store.js
import React, { createContext, useState } from "react";
export const AppContext = createContext();

export const AppProvider = ({ children }) => {
  const [count, setCount] = useState(0);
  const [theme, setTheme] = useState("light");

  const themes = {
    light: { backgroundColor: "white", color: "black" },
    dark: { backgroundColor: "black", color: "white" }
  };

  const toggleTheme = () => setTheme(theme === "light" ? "dark" : "light");

  return (
    <AppContext.Provider value={{ count, setCount, theme, toggleTheme, styles: themes[theme] }
      {children}
    </AppContext.Provider>
  );
};

// App.jsx
import React, { useContext } from "react";
import { AppContext, AppProvider } from "./store";
import Counter from "./Counter";
import ThemeSwitcher from "./ThemeSwitcher";

function AppContent() {
  const { styles } = useContext(AppContext);
  return (
    <div style={styles}>
      <h1>App Container</h1>
      <Counter />
      <ThemeSwitcher />
    </div>
  );
}

export default function App() {
  return (
```

```
    <AppProvider>
      <AppContent />
    </AppProvider>
  );
}

// Counter.jsx
import React, { useContext } from "react";
import { AppContext } from "./store";

export default function Counter() {
  const { count, setCount } = useContext(AppContext);
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(count - 1)}>Decrement</button>
    </div>
  );
}

// ThemeSwitcher.jsx
import React, { useContext } from "react";
import { AppContext } from "./store";

export default function ThemeSwitcher() {
  const { theme, toggleTheme } = useContext(AppContext);
  return (
    <div>
      <h2>Current Theme: {theme}</h2>
      <button onClick={toggleTheme}>Toggle Theme</button>
    </div>
  );
}
```

■ Interview Questions & Answers on useContext:

1. **Q1: What is the purpose of useContext in React?**
   A1: It allows components to consume values directly from a Context, avoiding prop drilling.
2. **Q2: What are the three main steps of using useContext?**
   A2: Create Context, Provide Context, and Consume Context.
3. **Q3: When should you use useContext instead of props?**
   A3: When you need to share global state across multiple components without passing props down many levels.
4. **Q4: How is useContext different from Redux?**
   A4: useContext is simpler and built into React for small-to-medium global state. Redux is more powerful, with middleware, devtools, and complex state handling.
5. **Q5: What happens if a component tries to useContext outside of its provider?**
   A5: It will return undefined and can lead to errors.